CARDIOSENSE

Project Guide

What the project does · How to pitch it to your guide · How to run it

Predicting Heart Diseases Using Machine Learning and Different Data Classification Techniques

[Your name] · [Month Year]

CONTENTS

1. THE PROJECT IN ONE PAGE 1

1.1 What Problem It Solves 1

1.2 What It Does 1

1.3 Key Numbers to Remember 1

2. HOW IT WORKS, IN PLAIN WORDS 3

3. WHAT MAKES THE PROJECT STAND OUT 4

4. PITCHING TO YOUR GUIDE 5

4.1 The 30-Second Pitch 5

4.2 The Two-Minute Version 5

4.3 Live Demo Script (about 5 minutes) 5

4.4 Questions Your Guide May Ask 6

4.5 Claims to Avoid 7

5. REQUIREMENTS TO RUN 8

6. STEP-BY-STEP: RUNNING THE PROJECT 9

6.1 Option A: Use the Ready Environment (Fastest) 9

6.2 Option B: Fresh Setup with Anaconda 9

6.3 Option C: Plain Python (No Anaconda) 9

6.4 Set the Keys 9

6.5 Start the App and Log In 10

6.6 Run the Automated Tests 10

6.7 Retrain the Models (Optional) 10

7. TROUBLESHOOTING 11

8. PROJECT FILES AT A GLANCE 12

9. BEFORE YOU SUBMIT 13

1. THE PROJECT IN ONE PAGE

> **In one sentence:** CardioSense predicts whether a patient has heart disease from 13 routine clinical measurements, and compares three ways of choosing which measurements matter (ANOVA, Chi-square and Mutual Information), using an honest evaluation and a web app that shows how each method sees the same patient.

1.1 What Problem It Solves

Heart disease is the leading cause of death worldwide, yet many warning signs are already in ordinary test results: blood pressure, cholesterol, heart rate during an exercise test, and so on. A model that reads those values can act as a cheap first screen. The research question is: **which of the 13 measurements do we really need, and does the way we choose them change how accurate the model is?**

1.2 What It Does

- **Predicts** heart disease for a patient, with a risk percentage, using the method you pick.
- **Compares** the same patient across all three methods side by side, and for real dataset patients shows the true diagnosis with ✓ Correct or ✗ Wrong.
- **Explains** each value: whether it points toward or away from heart disease, compared with the 303 patients in the data.
- **Shows the data and the science:** dataset statistics, the SMOTEENN resampling steps, feature-selection scores, an 11-classifier leaderboard, ROC curve and confusion matrix.
- **Keeps accounts safe:** email OTP sign-up, hashed passwords, login sessions, per-user history.

1.3 Key Numbers to Remember

Table 1.1: Numbers worth knowing by heart

What	Number
Patients in the dataset (UCI Cleveland)	303 (302 after removing one duplicate)
Training / test patients	241 / 61

What	Number
Measurements per patient	13 (fasting blood sugar is the only one no method picks)
Features kept: ANOVA / Chi-square / MI	8 / 11 / 10
Classifiers tried per method	11 (incl. Voting and Stacking ensembles)
Best model: ANOVA + KNN	85.5% cross-validated accuracy, AUC 0.87
Chi-square + Naive Bayes	84.6% CV; best recall 78.6%
MI + Naive Bayes	83.8% CV
Gap between the three methods	1.6 points, inside the ±2.3-point fold-to-fold spread: effectively tied
Labels found reversed in the downloaded file	303 of 303, corrected against UCI
Automated tests / responsive layout checks	All pass / 45 of 45

2. HOW IT WORKS, IN PLAIN WORDS

1. **Check the data.** The downloaded CSV had every "sick" and "healthy" label swapped. We proved it by matching all 303 patients to the original UCI file, then fixed it. *Analogy:* an answer key with every answer flipped; a student can still "score" well against it, but every conclusion is backwards.
2. **Lock away the exam.** 61 patients are set aside before anything learns from the data. They are the final exam; the model never sees them during training.
3. **Balance the class with SMOTEENN.** SMOTE invents a few realistic extra patients for the smaller group; ENN then removes confusing patients who look more like the other group. Done on training data only.
4. **Pick the measurements.** Each method scores the 13 features: ANOVA asks "do the averages differ between sick and healthy?", Chi-square asks "does the value depend on the diagnosis?", and Mutual Information asks "how much does knowing this value tell us?".
5. **Hold a tournament.** Eleven classifiers are trained on each method's features and scored by 5-fold cross-validation (five practice exams on the training data). The winner per method is saved.
6. **Take the final exam.** The saved models are tested once on the 61 locked-away patients. The web app then loads them to make predictions.

3. WHAT MAKES THE PROJECT STAND OUT

- **We caught a data error others miss.** Reversed labels don't lower accuracy, so they are invisible unless you check against the source and against medicine.
- **Honest evaluation.** The original notebooks claimed 100% because they oversampled before splitting (data leakage). We fixed the order and report cross-validated accuracy with its spread.
- **A fair comparison, not a forced winner.** We tried stronger stacking ensembles on all three methods the same way; none beat the simple winners, and we say so.
- **An app that teaches.** Same-patient comparison, real-diagnosis checks, "what stands out" explanations and live charts make the ML understandable.
- **Secure and tested.** Hashed passwords, OTP limits, sessions and environment-variable secrets; an automated suite and 45 layout checks.

4. PITCHING TO YOUR GUIDE

4.1 The 30-Second Pitch

> "We built CardioSense, a system that predicts heart disease from 13 routine clinical measurements and compares three feature-selection methods: ANOVA, Chi-square and Mutual Information. While checking the data we found that the popular copy of the dataset had every label reversed, and that the earlier 100% accuracy came from data leakage. After fixing both, our models reach about 84 to 86% cross-validated accuracy, and the three methods turn out to be effectively tied, with ANOVA doing it using only 8 measurements. The web app lets you run one patient through all three methods and see which got it right."

4.2 The Two-Minute Version

Table 4.1: Structure of a two-minute pitch

Part	Time	What to say
Problem	20 s	Heart disease is the top cause of death; routine test values already hold the warning signs. Which measurements matter?
Approach	30 s	UCI Cleveland data; three feature-selection methods; 11 classifiers each; SMOTEENN; best picked by cross-validation.
Surprise	25 s	Reversed labels in the dataset; 100% accuracy was leakage. We fixed both. This is the part guides remember.
Result	25 s	About 84–86% CV accuracy; methods tied within noise; ANOVA needs only 8 features; Chi-square catches the most sick patients.
App	20 s	Predict, compare the same patient across methods with ✓/✗, explanations, live charts, secure accounts.

4.3 Live Demo Script (about 5 minutes)

Table 4.2: Demo steps

#	Do this	Say this
1	Open Overview; scroll to the pipeline	"Seven steps, and the test set is split off before anything learns."
2	Open Dataset; show the SMOTEENN section	"SMOTEENN only touches training data, so the dataset still shows 303 real patients."
3	Predict → ANOVA → Random patient; hover two (i) icons	"Every input explains itself, with a fact from the data."

#	Do this	Say this
4	Click Predict	"Risk, verdict, and because this is a real patient, whether the model was right." Point at "What stands out".
5	Compare this patient with all 3 methods	"Same person, three methods. Each card shows its accuracy and what it ignores."
6	Insights → leaderboard, ROC, "effectively tied"	"All 11 classifiers; the best is chosen by cross-validation. The methods are within normal variation."
7	History; toggle dark mode	"Every prediction is saved per user."
8	Close	"It's a screening and study tool, not a diagnosis. Next: calibrated probabilities and data from more hospitals."

Tip. "Random patient" picks a different patient each time. Practise once beforehand. For a guaranteed sick patient, click Random until the note says "has heart disease".

4.4 Questions Your Guide May Ask

Table 4.3: Likely questions and answers

Question	Answer
Papers report 95–100% on this data. Why is yours lower?	Most of those oversample or select features before splitting, so test patients leak into training. Our original notebooks did this and got 100%. With the test set separated first, ~85% is the honest figure.
What is SMOTEENN, and why use it if the data is balanced?	SMOTE adds synthetic minority patients; ENN removes noisy ones. We measured it: a small, consistent gain (0.8–1.7 points). The default version hurt badly (73–76%) because unscaled cholesterol dominated; we tuned it.
Which feature-selection method is best?	They are effectively tied: 1.6 points apart, less than the ±2.3-point variation between folds. ANOVA is highest with the fewest features; Chi-square catches the most sick patients.
Why do KNN and Naive Bayes beat the ensembles?	With 241 training patients and 8–11 good features, simple models generalise well and ensembles overfit. Our stacking experiment on all three methods confirmed it.
Why choose the model by cross-validation, not the test score?	Choosing by test score makes the test set part of training. With 61 test patients, one patient is 1.6 points, so a test-set winner is often luck.
How do you know the labels are right now?	We matched every row to UCI's processed.cleveland.data on eight columns; all 303 matched and now all 303 labels agree. Clinical signs also point the right way.
Why not use more data?	The other UCI hospitals are missing most of the vessel and thallium values every model relies on. Adding them would add noise. It is listed as future work.
What does the risk percentage mean?	The model's probability. Naive Bayes and KNN are overconfident, so we show "<1%" or ">99%" for extremes. Calibration is the next step.

Question	Answer
How does “What stands out” work?	It compares each value with the dataset (e.g. 77% of patients with exercise angina have heart disease). It describes the data, not the model’s internal reasoning.
Is it secure?	Passwords hashed with scrypt; email password in environment variables; OTP expires in 5 minutes with 5 tries; sessions on every page; login over POST.
What did you add beyond the original notebooks?	Label fix, leakage-free pipeline, tuned SMOTEENN, best-of-11 selection, CV with spread, stacking experiment, the whole secure web app, tests.
Could a hospital use this?	Not as it stands: 302 patients from one hospital, not clinically validated. It is a study and screening demonstrator.

4.5 Claims to Avoid

- “It diagnoses heart disease.” Say: it predicts risk from a study dataset; a doctor must confirm.
- “It is 100% / 95% accurate.” Say: about 84–86% cross-validated.
- “Mutual Information (or any method) is the best.” Say: the three are effectively tied.
- “The explanation shows why the model decided.” Say: it shows what the data says about each value.
- “It works for any population.” Say: it was trained on one Cleveland cohort.

5. REQUIREMENTS TO RUN

Table 5.1: What you need

Item	Requirement
Computer	Windows 10/11, macOS or Linux; 4 GB RAM or more; about 2 GB free disk
Python	3.10 or newer (tested on 3.13). Anaconda works well on Windows
Packages	Listed in requirements.txt (app) and requirements-train.txt (retraining); installed with pip
Browser	Chrome, Edge, Firefox or Safari
Internet	Needed to install packages and to send OTP emails
Gmail app password	Only needed for new sign-ups (OTP emails). Create at Google Account → Security → 2-Step Verification → App passwords

6. STEP-BY-STEP: RUNNING THE PROJECT

6.1 Option A: Use the Ready Environment (Fastest)

An environment called `heart313` with everything installed already exists on this computer. Open the **Anaconda Prompt** and run:

```
conda activate heart313
cd "C:\Users\bonda\OneDrive\Desktop\Project 4-1\4-1 V2\Predicting Heart Diseases Using Machine Learning and Different Data Classification Techniques"
```

Then set the keys (6.4) and start the app (6.5).

6.2 Option B: Fresh Setup with Anaconda

```
conda create -n heart313 python=3.13 -y
conda activate heart313
cd "C:\Users\bonda\OneDrive\Desktop\Project 4-1\4-1 V2\Predicting Heart Diseases Using Machine Learning and Different Data Classification Techniques"
pip install -r requirements.txt
```

If `conda create` fails with an SSL or "cannot find Python 3.13" error, update conda first with `conda update -n base conda`, or use Option C.

6.3 Option C: Plain Python (No Anaconda)

```
cd "C:\Users\bonda\OneDrive\Desktop\Project 4-1\4-1 V2\Predicting Heart Diseases Using Machine Learning and Different Data Classification Techniques"
py -3.13 -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

6.4 Set the Keys

These last only for the current window, so set them each time you open a new one. In the **Anaconda Prompt or Command Prompt**:

```
set SECRET_KEY=any-long-random-text
set SMTP_USER=your-gmail@gmail.com
set SMTP_PASS=your-16-letter-app-password
```

In **PowerShell**:

```
$env:SECRET_KEY = "any-long-random-text"
$env:SMTP_USER  = "your-gmail@gmail.com"
$env:SMTP_PASS  = "your-16-letter-app-password"
```

`SECRET_KEY` keeps people logged in across restarts. `SMTP_USER` and `SMTP_PASS` are needed only to register new accounts. Type the app password without spaces.

6.5 Start the App and Log In

```
python app.py
```

1. Wait for `Running on http://127.0.0.1:5000`, then open that address in your browser.
2. Log in with an existing account (your original password still works; the app hashed it automatically), or click **Get started** to register: you will receive a 6-digit code by email.
3. Stop the app with **Ctrl + C** in the terminal.

6.6 Run the Automated Tests

```
python test_app.py
```

It takes about 5 seconds and should print `all checks passed` and `model checks passed`. It uses a throwaway database, so your real accounts are never touched.

6.7 Retrain the Models (Optional)

```
pip install -r requirements-train.txt
python train.py
python experiment_stacking.py
```

`train.py` takes about 3 minutes and rewrites the three model files and `models/metrics.json`; `experiment_stacking.py` takes about a minute. Restart the app afterwards; every chart updates automatically.

7. TROUBLESHOOTING

Table 7.1: Common problems

What you see	Why	Fix
No module named 'numpy._core'	Old environment with numpy 1.x (e.g. the old heart env on Python 3.10)	conda activate heart313, or create a new environment (6.2)
No module named 'flask'	Packages not installed in the active environment	Activate the environment, then pip install -r requirements.txt
Python version errors during pip install	Python older than 3.10	Create the environment with python=3.13
“Email is not configured on the server”	SMTP_USER / SMTP_PASS not set in this window	Set them (6.4) and restart the app
“Could not send the OTP email”	Wrong app password, or Gmail blocked the sign-in	Create a new app password; check internet
Logged out after every restart	SECRET_KEY not set	Set SECRET_KEY (6.4)
“Address already in use” on port 5000	Another copy of the app is running	Close the other terminal, or press Ctrl + C there
conda: SSL / HTTP error	Old conda (4.7) or proxy	conda update -n base conda, or use Option C
Page looks unstyled	Browser cached an old file	Press Ctrl + F5

8. PROJECT FILES AT A GLANCE

Table 8.1: Where things are

File / folder	What it is
app.py	The web application: pages, input rules, accounts, predictions
train.py	Retrains everything; writes models/*.sav and models/metrics.json
experiment_stacking.py	Tries three extra stacking ensembles on all methods
test_app.py	Automated tests
templates/, static/css/app.css, static/js/app.js	Pages, styling (light and dark) and interactions
archive/heart.csv	The corrected dataset (original kept as heart_kaggle_original.csv)
models/	Saved models, metrics.json, stacking_experiment.json
signup.db	User accounts and prediction history
README.md, TEST_REPORT.md, SECURITY_REPORT.md	Documentation, test results, security review
../reports/	This guide and the project report
../presentation/screenshots/	App screenshots for slides and the report

9. BEFORE YOU SUBMIT

- Fill in every [bracketed] placeholder in the project report, the slides (cover and last slide) and this guide: names, register numbers, guide, college, dates.
- Revoke the old Gmail app password that used to be in app.py, and create a new one for the demo.
- Check each reference in the project report against the original source (title, authors, year, pages).
- Check the WHO figure (17.9 million deaths a year) against the current fact sheet.
- Rehearse the demo once with the app running; run python test_app.py the same day.
- Share the slide deck from its Share menu if your guide or team should open it, and download it as PowerPoint or PDF.